

MapReduce

COMPCSI 752: Big Data Management

University of Auckland

Credits to Ninh Pham for the Slides

Slides are collected and edited from <http://webdam.inria.fr/Jorge/index9213.html>
and <https://homepages.cwi.nl/~boncz/bads/mapreduce.shtml>

Outline

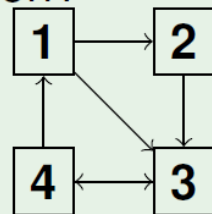
- The MapReduce framework
 - MapReduce
 - HDFS
 - Apache Hadoop
 - Apache Spark
- MapReduce algorithms:
 - PageRank
 - Similarity join

PageRank computation

- **PageRank**: **importance score** for nodes in a graph, used for ranking query results of Web search engines:
 - Compute $\mathbf{M}$.
 - Let $\mathbf{v}_0$ be the uniform vector of sum 1, $\mathbf{v} = \mathbf{v}_0$
 - Repeat $\mathbf{N}$ times:
 - Set $\mathbf{v} := \mathbf{M}\mathbf{v}$

Exercise

Express PageRank computation as a MapReduce problem. Main program? *map* and *reduce* functions? *combiner* function?

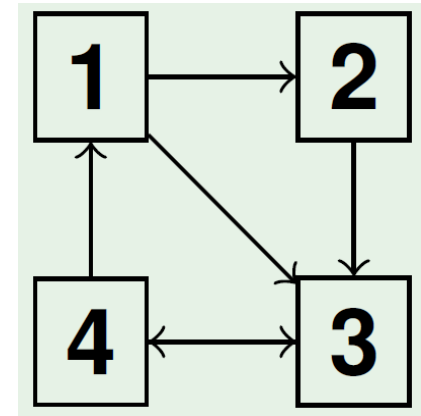


Illustrate on this graph.

PageRank exercise

- Setup:

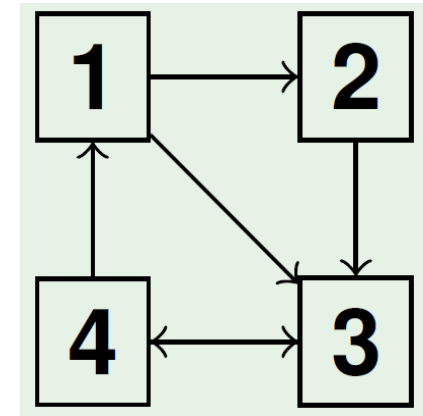
$$\mathbf{M} = \begin{bmatrix} 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & \frac{1}{2} \\ 0 & 0 & 1 & 0 \end{bmatrix}, \mathbf{v}_0 = \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}$$



- Results of the first and second rounds:

$$\mathbf{v}_1 = \mathbf{M}\mathbf{v}_0 = \begin{bmatrix} 1/8 \\ 1/8 \\ 1/2 \\ 1/4 \end{bmatrix}, \mathbf{v}_2 = \mathbf{M}\mathbf{v}_1 = \begin{bmatrix} 2/16 \\ 1/16 \\ 5/16 \\ 8/16 \end{bmatrix},$$

PageRank exercise



- Setup:

$$\mathbf{M}\mathbf{v}_0 = \begin{bmatrix} 0 & 0 & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 & 0 \\ \frac{1}{2} & 1 & 0 & \frac{1}{2} \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix} = \begin{bmatrix} 1/8 \\ 1/8 \\ 1/2 \\ 1/4 \end{bmatrix}$$

- New set up:

$$\mathbf{M}\mathbf{v}_0 = \begin{bmatrix} 0 \\ 1/2 \\ 1/2 \\ 0 \end{bmatrix} \frac{1}{4} + \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \frac{1}{4} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \frac{1}{4} + \begin{bmatrix} 1/2 \\ 0 \\ 1/2 \\ 0 \end{bmatrix} \frac{1}{4}$$

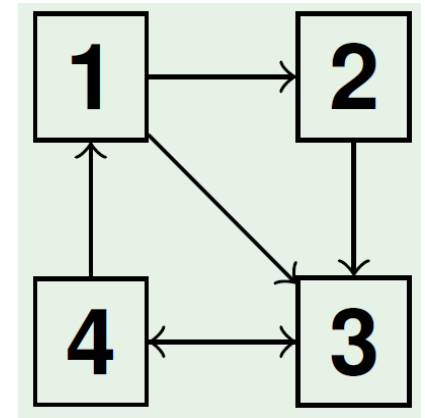
Node A

Node B

Node C

Node D

MapReduce: PageRank



Node

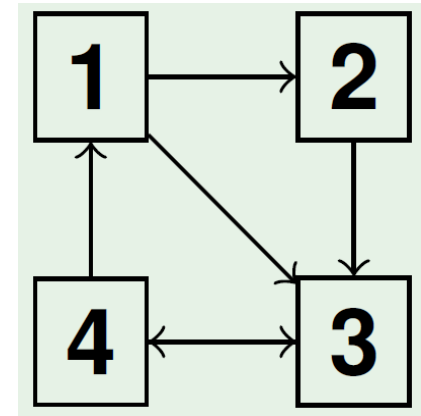
Algorithm 1 `map(key: [url, pagerank], value: outlink_list)`

```
1: for each outlink in outlink_list do
2:   emit(key: outlink, value: pagerank/size(outlink_list))
3: end for
4: emit(key: url, value: outlink_list)
```

Node

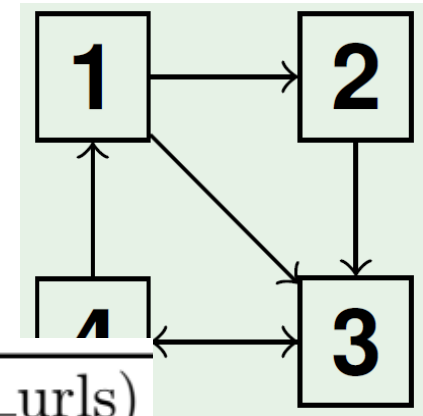
- We emit **both** pagerank contributions and the outlink_list since we will use the outlink_list for the next map-reduce

Map: Example



- Assume that we have 1 mapper for each url
- Map inputs:
 - Node 1: (key = [1, 1/4], value = [2, 3])
 - Node 2: (key = [2, 1/4], value = [3])
 - Node 3: (key = [3, 1/4], value = [4])
 - Node 4: (key = [4, 1/4], value = [1, 3])
- Map outputs:
 - Node 1: (2, 1/8), (3, 1/8), (1, [2, 3])
 - Node 2: (3, 1/4), (2, [3])
 - Node 3: (4, 1/4), (3, [4])
 - Node 4: (1, 1/8), (3, 1/8), (4, [1, 3])

MapReduce: PageRank



Algorithm 2 `reduce(key: url, value: list_pagerank_or_urls)`

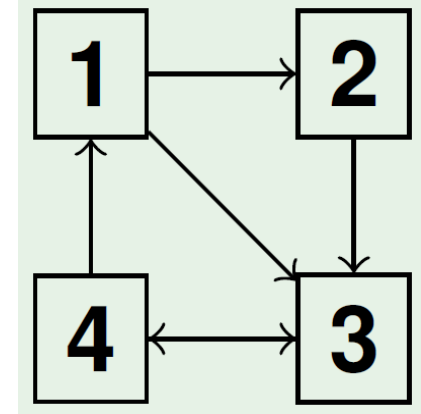
```
1: outlink_list = [ ]
2: pagerank = 0
3: for each pagerank_or_urls in list_pagerank_or_urls do
4:   if is_list(pagerank_or_urls) then
5:     outlink_list = pagerank_or_urls
6:   else
7:     pagerank += pagerank_or_urls
8:   end if
9: end for
10: pagerank = d * pagerank + (1 - d)/n
11: emit(key: [url, pagerank], value: outlink_list)
```

Node

Check urls or pagerank

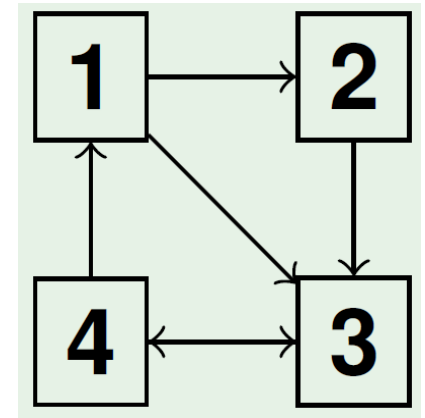
- urls is a list of nodes
- pagerank is a real value

Reduce: Example



- Assume that we have 1 reducer for each url, $d = 1$
- Reduce inputs:
 - Node 1: $(1, [1/8, [2, 3]])$
 - Node 2: $(2, [1/8, [3]])$
 - Node 3: $(3, [1/8, 1/4, 1/8, [4]])$
 - Node 4: $(4, [1/4, [1, 3]])$
- Reduce outputs:
 - Node 1: $([1, 1/8], [2, 3])$
 - Node 2: $([2, 1/8], [3])$
 - Node 3: $([3, 1/2], [4])$
 - Node 4: $([4, 1/4], [1, 3])$

PageRank example



- Reduce outputs:

- Node 1: $([1, 1/8], [2, 3])$
- Node 2: $([2, 1/8], [3])$
- Node 3: $([3, 1/2], [4])$
- Node 4: $([4, 1/4], [1, 3])$

- Verifications:

- $\mathbf{v}_1 = \mathbf{M}\mathbf{v}_0 = \begin{bmatrix} 1/8 \\ 1/8 \\ 1/2 \\ 1/4 \end{bmatrix}, \mathbf{v}_2 = \mathbf{M}\mathbf{v}_1 = \begin{bmatrix} 2/16 \\ 1/16 \\ 5/16 \\ 8/16 \end{bmatrix}, \dots$

Outline

- The MapReduce framework
 - MapReduce
 - HDFS
 - Apache Hadoop
 - Apache Spark
- Applications:
 - PageRank
 - Similarity join

Inverted index solution

Term

Documents

family	(d1, .13), (d3, .13), (d6, .08), (d5, .07)
football	(d4, .47)
jaguar	(d1, .04), (d2, .04), (d3, .04), (d4, .04), (d6, .04), (d5, .02)
new	(d2, .24), (d1, .20), (d5, .10)
rule	(d6, .28)
us	(d4, .30), (d5, .15)
world	(d1, .47)
...	

family: (d1, d3), (d1, d6), (d1, d5), (d3, d6), (d3, d5), (d6, d5)

...

new: (d2, d1), (d2, d5), (d1, d5)

...

Candidate
pairs

MapReduce: Index construction

- Map input ($\mathbf{i}, \mathbf{d_i}$):
 - For each document $\mathbf{i}$, we have a list $\mathbf{d_i}$ consisting of terms $\mathbf{t}$ and its weight $\mathbf{d_i[t]}$
- Map output ($\mathbf{t}, [\mathbf{i}, \mathbf{d_i[t]}]$):
 - For each term $\mathbf{t}$ in document $\mathbf{i}$, we output $\text{key} = \mathbf{t}$, $\text{value} = [\mathbf{i}, \mathbf{d_i[t]}]$ as a pair of document ID and the weight of the term $\mathbf{t}$
- Reduce: do nothing

$$\begin{aligned}\text{Map :} \quad & \langle i, d_i \rangle \rightarrow [\langle t, \langle i, d_i[t] \rangle \rangle \mid d_i[t] > 0] \\ \text{Reduce :} \quad & \langle t, [\langle i, d_i[t] \rangle, \langle j, d_j[t] \rangle, \dots] \rangle \rightarrow \\ & [\langle t, [\langle i, d_i[t] \rangle, \langle j, d_j[t] \rangle, \dots] \rangle]\end{aligned}$$

MapReduce: Similarity join

- Map input $(\mathbf{t}, [(i, d_i[\mathbf{t}]), (j, d_j[\mathbf{t}]), \dots])$:
 - For each term $\mathbf{t}$, we have a posting list of document ID consisting of $\mathbf{t}$ and its weight
- Map output $((i, j), [d_i[\mathbf{t}] * d_j[\mathbf{t}]])$:
 - For each pair of documents (i, j) where $i < j$, we compute the product $d_i[\mathbf{t}] * d_j[\mathbf{t}]$ as the contribution of the term $\mathbf{t}$ to their inner product

$$\begin{aligned} \text{Map : } & \langle t, [\langle i, d_i[t] \rangle, \langle j, d_j[t] \rangle, \dots] \rangle \rightarrow \\ & [\langle \langle i, j \rangle, w = d_i[t] \cdot d_j[t] \rangle \mid i < j] \\ \text{Reduce : } & \langle \langle i, j \rangle, W = [w_0, \dots, w_{|W|}] \rangle \rightarrow \\ & [\langle \langle i, j \rangle, \sigma(d_i, d_j) = \sum_{w \in W} w \rangle] \end{aligned}$$

MapReduce: Similarity join

- Reduce input $((i, j), [w_0, w_1, \dots, w_{|W|}])$:
 - For each pair of documents (i, j) where $i < j$, we have a list of values, each corresponding to the contribution of the common term t
- Reduce output $((i, j), \sigma(i, j) = w_0 + w_1 + \dots + w_{|W|})$:
 - For each pair of documents (i, j) where $i < j$, we compute the inner product σ by the sum of all term contributions and decide the matching

$$\begin{aligned} \text{Map : } & \langle t, [\langle i, d_i[t] \rangle, \langle j, d_j[t] \rangle, \dots] \rangle \rightarrow \\ & [\langle \langle i, j \rangle, w = d_i[t] \cdot d_j[t] \rangle \mid i < j] \\ \text{Reduce : } & \langle \langle i, j \rangle, W = [w_0, \dots, w_{|W|}] \rangle \rightarrow \\ & [\langle \langle i, j \rangle, \sigma(d_i, d_j) = \sum_{w \in W} w \rangle] \end{aligned}$$

Prefix-filtering solution

- Observation:
 - We do not want to index all documents for each term
- Do not want to compute $\sigma(\mathbf{x}, \mathbf{y})$ if
 - $\sigma(\mathbf{x}, \mathbf{y}) = x_1y_1 + x_2y_2 + \dots + x_ky_k + x_{k+1}y_{k+1} + \dots + x_dy_d < \text{thres}$
- Define vector $\mathbf{m}$ containing the **maximum** value for each dimension

$$\bullet \quad \underbrace{x_1m_1 + x_2m_2 + \dots + x_km_k}_{\text{Maximum contribution from dimensions 1 to } k} + \underbrace{x_{k+1}m_{k+1} + \dots + x_dm_d}_{\text{If no intersection between } \mathbf{x} \text{ and } \mathbf{y} \text{ on dimensions } k+1 \text{ to } d \text{ then } \text{sim}(\mathbf{x}, \mathbf{y}) < \text{thres}}$$

$< \text{thres}$

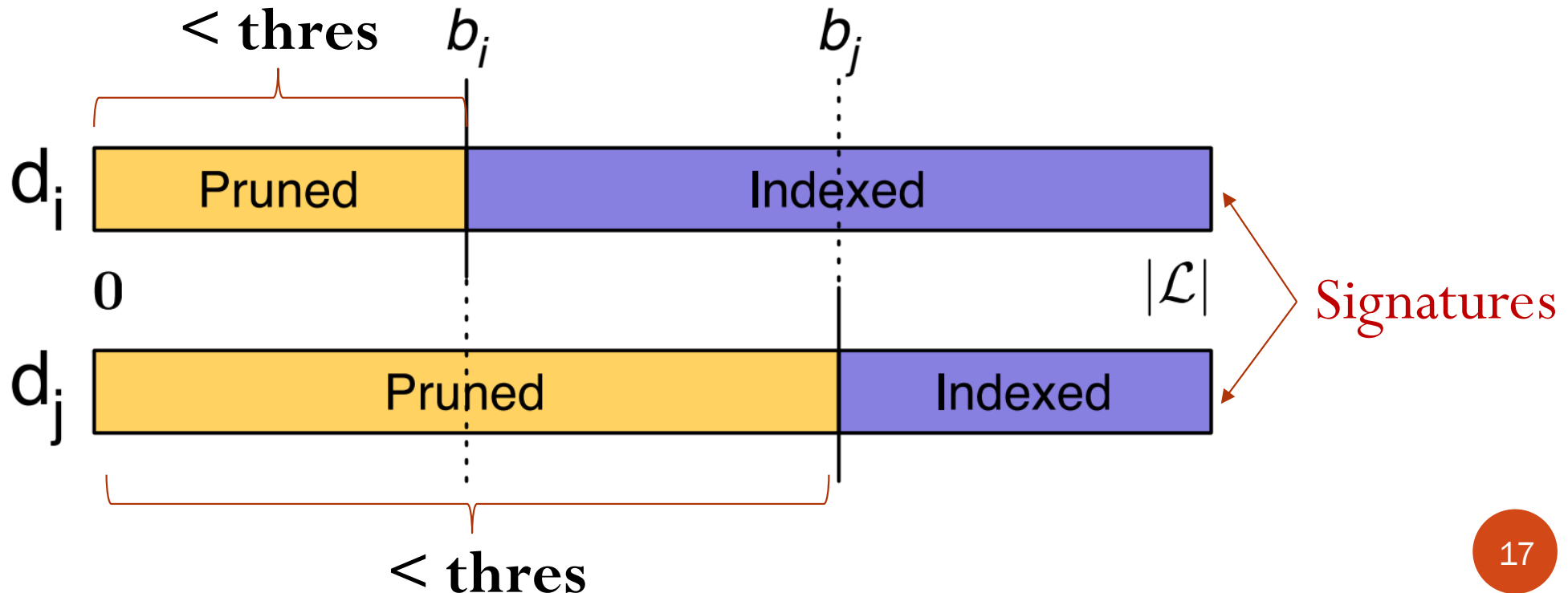
$\geq \text{thres}$

Maximum contribution
from dimensions 1 to k

If no intersection between
 $\mathbf{x}$ and $\mathbf{y}$ on dimensions $k+1$ to
 d then $\text{sim}(\mathbf{x}, \mathbf{y}) < \text{thres}$

Prefix-filtering solution

- $\mathbf{b}_i$ ($\mathbf{b}_j$) is the **maximum** position of $\mathbf{d}_i$ ($\mathbf{d}_j$) s.t. the **maximum** of partial sum from dimensions 1 to $\mathbf{b}_i$ ($\mathbf{b}_j$) is still smaller than **thres**
- We only index the **signatures** of $\mathbf{d}_i$ and $\mathbf{d}_j$



MapReduce: Prefix-filtering

- Map input ($\mathbf{i}, \mathbf{d}_i$):
 - For each document $\mathbf{i}$, we keep a list $\mathbf{d}_i$ consisting of terms $\mathbf{t}$ and its weight $\mathbf{d}_i[\mathbf{t}]$
- Map output ($\mathbf{t}, \mathbf{d}_i$) for each term $\mathbf{t}$ in the signature $S(\mathbf{d}_i)$:
 - For each term $\mathbf{t}$ in the signature, we output the list $\mathbf{d}_i$ (whole document)

Map : $\langle i, d_i \rangle \rightarrow [\langle t, d_i \rangle \mid t \in S(d_i)]$

Reduce : $\langle t, D = [d_0, \dots, d_{|D|}] \rangle \rightarrow$
 $[\langle i, j \rangle, \sigma(d_i, d_j) \mid$
 $d_i, d_j \in D \wedge \sigma(d_i, d_j) \geq \text{thres}]$

MapReduce: Prefix-filtering

- Reduce input $(\mathbf{t}, [\mathbf{d}_i, \mathbf{d}_j, \dots])$:
 - For each term $\mathbf{t}$, we have a list of list $\mathbf{d}_i$ of the document $\mathbf{i}$ which contains $\mathbf{t}$ in their signature
- Reduce output $((\mathbf{i}, \mathbf{j}), \sigma(\mathbf{d}_i, \mathbf{d}_j))$ if $\sigma(\mathbf{d}_i, \mathbf{d}_j) \geq \text{thres}$:
 - For each pair of documents $(\mathbf{i}, \mathbf{j})$ where $\mathbf{i} < \mathbf{j}$, we compute their inner product σ using $\mathbf{d}_i$ and $\mathbf{d}_j$ and decide the matching

Map : $\langle i, d_i \rangle \rightarrow [\langle t, d_i \rangle \mid t \in S(d_i)]$

Reduce : $\langle t, D = [d_0, \dots, d_{|D|}] \rangle \rightarrow$
 $[\langle i, j \rangle, \sigma(d_i, d_j) \mid$
 $d_i, d_j \in D \wedge \sigma(d_i, d_j) \geq \text{thres}]$

Homework

- Writing the Hadoop MapReduce programs to compute PageRank and Similarity Join (both versions)
 - Input of PageRank is the same as the PageRank lecture's homework
 - Input of Similarity Join is the same as the Data Cleaning lecture's homework
- Verify the accuracy of the results between the MapReduce version and the non-MapReduce version
- **Challenges:** Can we adapt these solutions for other similarity measures, e.g. Jaccard similarity, containment similarity?